

Conceptual Modelling: Entity-Relationship Diagrams

Databases and Software Architecture – Aalborg University

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.

Conceptual Modelling with ER Diagrams

Last time we built tables directly in PostgreSQL. Today we step back and **design** a database before writing a single `CREATE TABLE` – using an Entity-Relationship (ER) diagram to sketch the domain first, in a tool like draw.io or PlantUML. A later lecture turns today's diagrams into relational schemas.

Agenda

Six parts, about 105 minutes: a recap, core vocabulary (entities, attributes), relationships (cardinality, participation), a break, weak entities, then UML comparison and a live demo.

- Recap: keys and integrity, and why we model before we build
- Entities and attributes (simple, composite, multi-valued, derived)
- Relationships: cardinality and participation
- Break
- Weak entities and identifying relationships
- ER vs. UML class diagrams, live demo, and your in-class exercise

Recap and Motivation

Recap: Keys and Integrity

Quick recall before we build on it:

- **Primary key** – uniquely identifies each row
- **Foreign key** – points at another table's primary key (or a unique column);
how relationships are represented
- **Domain integrity** – values match their declared type
- **Entity integrity** – every primary key is unique and non-null
- **Referential integrity** – every foreign key matches an existing row, or is NULL

Today's question: **before** writing tables and foreign keys, how do we find the “right” tables?

Why Model Before We Build?

Last lecture's exercise raised open questions you answered by best judgement
– can a book have several authors? Can an order contain several books? Today gives a disciplined way to answer them.

- Jumping straight to `CREATE TABLE` works for toy examples, but real domains have optional properties, relationship data, and dependencies – skipping design tends to produce schemas that break on edge cases
- An **ER diagram** is a DBMS-independent blueprint of entities, attributes, and relationships – readable by a domain expert who has never heard of SQL
- It is exactly what a later lecture translates into `CREATE TABLE` statements

Entities and Attributes

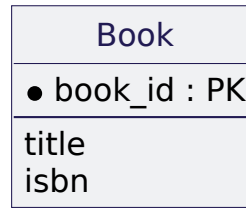
What Is an Entity?

- **Entity**: a distinguishable real-world thing to record data about – a specific book, author, or member
- **Entity type** (entity set): a collection of entities sharing attributes – Book is an entity type, one shelved copy of *Database System Concepts* is an entity belonging to it
- Exactly parallel to **table** and **row** in the relational model – same idea, one abstraction level earlier

Running example all lecture: a **library system** with Book, Copy, Author, Member – also the domain of today's in-class exercise.

Entity Notation

An entity type is a rectangle labelled with its name. We use **crow's foot** style throughout – same as draw.io and PlantUML – with key attributes listed inside the box, like a table definition.



A strong entity: a single-bordered rectangle, name on top, key attributes listed below a rule

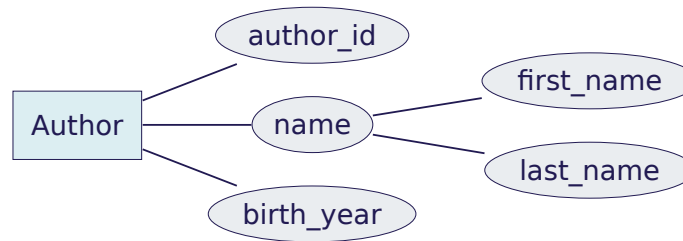
A second box style appears later – **heavier border, distinct colour** – marking a **weak entity** that cannot stand on its own.

Attributes: Simple (Atomic)

An **attribute** is a property of an entity type. A **simple** (atomic) attribute cannot be split further – `Book.isbn`, `Book.publication_year`, `Member.email`. Most attributes are this kind: no special notation, just listed in the box (or a plain oval, in the classical notation).

Attributes: Composite

A **composite** attribute groups several simple attributes under one name. Author.name decomposes into first_name and last_name, which we often need to query or sort on separately.

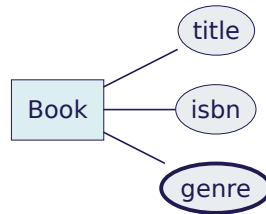


Classical (Chen-style) attribute notation: name decomposes into first_name and last_name

In our crow's-foot boxes, this is just name (first_name, last_name) – the oval-and-branches picture is mainly for **teaching** the concept.

Attributes: Multi-Valued

A **multi-valued** attribute can hold several values per instance. Book.genre is multi-valued – a novel can be both “Fantasy” and “Young Adult.” Classical notation: a heavier (or double) outlined oval.



genre is multi-valued – shown with a heavier outline – because a book may have several

Strong hint for the relational model: it cannot become a single column – a later lecture makes it its own table.

Attributes: Derived

A **derived** attribute is computed from other attributes on demand, never stored. Classic example: `Person.age`, derived from `birth_date` and today's date – never stale. Classical notation: a dashed oval.

Our own example lives on a **relationship**, not an entity: `Borrows.is_overdue`, derived from `due_date` and `return_date` – drawn properly in a few slides.

Keys, Revisited: Superkey and Candidate Key

Recall from last lecture, for Student:

- **Superkey**: any attribute set that uniquely identifies an instance – {student_id} is one, and so is {student_id, email} (redundant, but still unique)
- **Candidate key**: a **minimal** superkey – drop any attribute and uniqueness breaks. {student_id} qualifies; {student_id, email} does not, since {student_id} alone already works

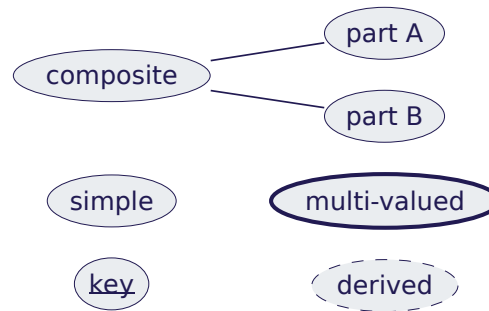
From Candidate Key to Identifying Attribute

In an ER diagram, the **identifying attribute** we underline inside an entity box is simply the candidate key the designer has chosen as that entity type's primary key.

- `book_id`, `author_id`, `member_id` – one designer-chosen candidate key each, underlined
- A weak entity, after the break, does not get this on its own – more shortly
- Later in the course: this chosen candidate key becomes the table's actual PRIMARY KEY

Attribute Notation Legend

The full classical vocabulary in one place – you'll see all four shapes again today.



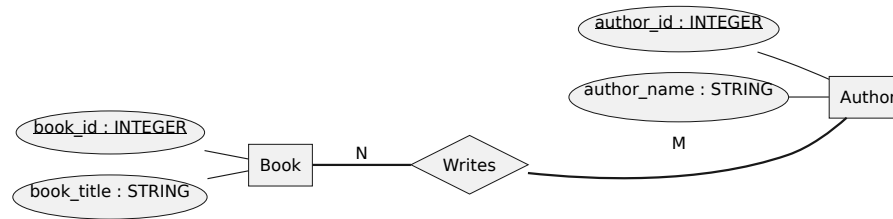
Simple, key (underlined), composite (with parts), multi-valued (heavier outline), and derived (dashed outline)

Two Notations for Entities: Chen vs. Crow's Foot

The course reading (Silberschatz et al., ch. 6) uses **Chen notation** – relationships as diamonds. Every tool you will actually draw with – draw.io, PlantUML – uses **crow's foot** instead, no diamond at all. **We use crow's foot for the rest of this course.**

Chen notation	Crow's foot notation
Relationship: its own diamond shape	Relationship: a direct line, no diamond
Attributes: ovals branching off entities	Attributes: listed inside the entity box
This course's textbook	draw.io, PlantUML, and this deck

The Same Fact, Two Notations: Writes



Chen notation: Writes as its own diamond; book_id/book_title and author_id/author_name as ovals branching off – underlined ovals mark each key

- Same fact we formalize next: Writes is M:N, total participation on both sides
- Compare the diamond-and-ovals here to the box-and-line style we use for the rest of this deck

Relationships, Cardinality, and Participation

What Is a Relationship?

- **Relationship**: an association between entity instances – Writes links Book and Author; Borrows links Member and Copy
- **Relationship type** (set): all instances of the same kind, just like entity type/entity – every row of “who wrote what” belongs to Writes

Relationships are first-class citizens, on equal footing with entities – hence Chen’s diamond shape and crow’s foot’s own line and label.

Relationships Can Involve More Than Two Entities

- **Binary** – two entity types, e.g. Book–Author – the only kind we build in depth today
- **Unary** – an entity related to itself, e.g. Employee “manages” Employee
- **Ternary** – three entity types at once, e.g. Supplier–Part–Project
- Binary relationships cover the overwhelming majority of real designs, and everything in this course

Cardinality: What It Answers

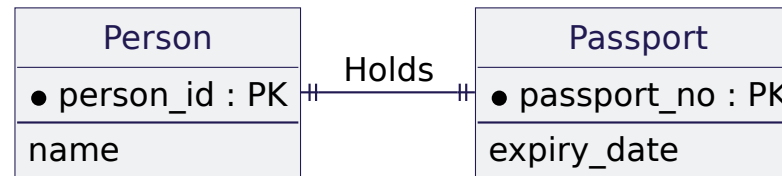
Cardinality answers: “for one instance of A, how many instances of B can it relate to, and vice versa?” It is the single most consequential design decision – later in the course it determines whether a relationship becomes a foreign key or a whole new table.

There are three cardinality patterns to know: **one-to-one** (1:1), **one-to-many** (1:N), and **many-to-many** (M:N).

Cardinality: One-to-One (1:1)

One instance of A relates to at most one instance of B, and vice versa.

Example: a person has at most one passport; a passport belongs to exactly one person.

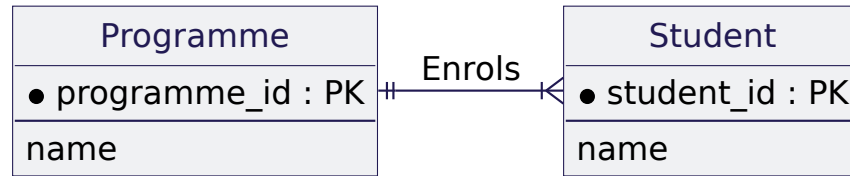


1:1 - a double bar at both ends of the line, since both sides here are mandatory

(The double bar here does two jobs – “at most one” and “mandatory” – separated properly once we reach participation.) 1:1 is the rarest pattern in practice – often a sign the two entity types should just be merged.

Cardinality: One-to-Many (1:N)

One instance of A relates to many instances of B; each B relates to only one A. The most common cardinality – you already met one, last lecture:

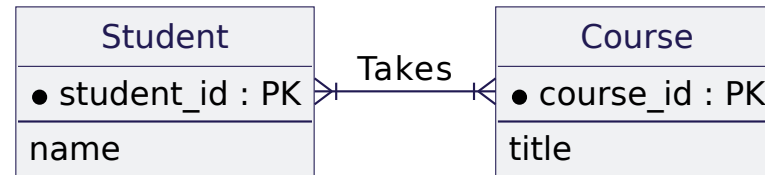


1:N – a double bar on the “one” side, a crow’s foot on the “many” side

One programme has many students; each student belongs to exactly one. This is exactly `Student.programme_id REFERENCES Programme.programme_id` – the crow’s foot side is where the foreign key will live, later in the course.

Cardinality: Many-to-Many (M:N)

Many instances of A relate to many instances of B, in both directions. Example: a student enrolls in several courses; a course has several students.

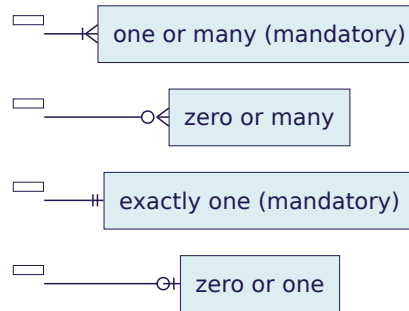


M:N – a crow's foot (plus a mandatory bar, in this example) at both ends

Our own example: Book and Author via Writes – a book may have several authors, an author several books. Built properly later this lecture.

Crow's Foot Notation: Reading the Symbols

This and the next few slides are a self-contained crow's-foot reference – come back anytime. Each symbol sits where the line touches an entity box, showing that entity's multiplicity, counted per one instance on the other end.



Circle = "zero" is allowed; bar = "exactly one"; crow's foot = "many"

Two symbols combine at each end: **circle** (zero allowed) or **bar** (one required) on the inside; nothing or a **crow's foot** (many allowed) on the outside. The inner circle-or-bar is exactly **participation** – next.

Participation: Total vs. Partial

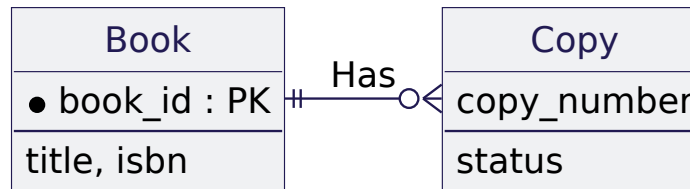
Participation answers a different question: “must every instance take part in this relationship at all?”

- **Total**: yes – every instance must relate to at least one instance on the other side
- **Partial**: no – an instance can exist without ever appearing in the relationship

Chen: total participation is a double line to the relationship diamond. Crow’s foot: the bar-vs-circle choice you just saw – bar means the far entity is mandatory, circle means optional.

Participation Illustrated: Book and Copy

Book and Copy, related by Has. Every Copy needs a Book – **total** participation. A Book can sit in the catalogue with zero copies – **partial** participation.

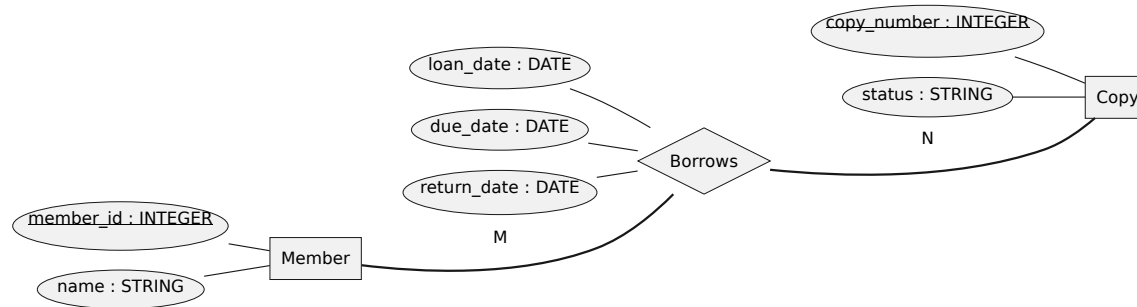


The double bar next to Book forces total participation of Copy; the circle next to Copy allows Book to be partial

Read end to end: near Book, the double bar (“1!”) says every copy has exactly one book – making Copy total. Near Copy, the circle-plus-fork (“n?”) says a book can have zero or many copies – making Book partial.

Attributes on Relationships

Sometimes a fact belongs to the **relationship itself**, not either entity. Borrows (Member–Copy): `loan_date`, `due_date`, `return_date` describe one loan event, not a permanent property of either side. Chen notation draws this directly: an attribute oval can hang off a relationship diamond exactly as it hangs off an entity.

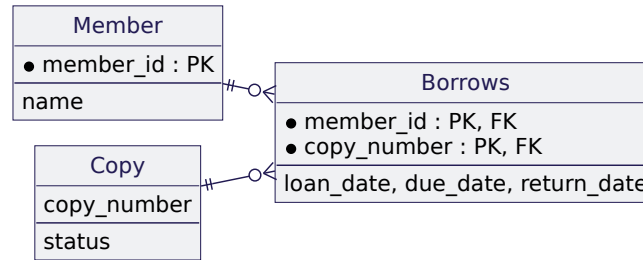


Chen notation: `loan_date`, `due_date`, and `return_date` as ovals branching off the Borrows diamond – attributes of the relationship itself, not of Member or Copy

Crow's foot notation has no equivalent symbol for an attribute on a line – next slide shows how it's handled instead.

The Same Fact, in Crow's Foot: Borrows Becomes an Entity

Crow's foot has no way to attach an attribute directly to a relationship **line**. The fix: promote the relationship to its own entity – Borrows gets a box, a key made of the two foreign keys it connects, and `loan_date/due_date/return_date` as ordinary attributes. The M:N relationship becomes two 1:N relationships instead.



Borrows promoted to its own entity, keyed by (member_id, copy_number), with loan_date/due_date/return_date as its attributes

This is exactly what happens when translating to SQL – a many-to-many relationship always becomes its own table (a later lecture covers this in full). The compact crow's-foot diagrams later in this deck take a shortcut and just note the attributes on the line instead.

Derived Attribute in Practice: `is_overdue`

The derived attribute we promised earlier. `is_overdue` is computed from `due_date` vs. today's date – only meaningful while `return_date` is still NULL.



Chen notation: `is_overdue` – dashed outline – is never stored; it is recomputed on demand

`is_overdue` → `due_date < today()` AND `return_date IS NULL`. A genuinely useful pattern: a value that can go stale is a bug magnet; computing it on the fly is always correct.

Relationships: Recap

Before the break, the vocabulary so far:

- **Relationship** → an association between entity instances; a **relationship type** groups similar ones
- **Cardinality** → 1:1, 1:N, or M:N – how many instances relate to how many
- **Participation** → total (mandatory, bar) vs. partial (optional, circle)
- **Attributes on relationships** → facts that belong to the association itself, like `Borrows.loan_date`
- **Derived attributes** → computed on demand, never stored, like `Borrows.is_overdue`

Break



Stretch, grab water, ask now if cardinality or participation feels shaky. Next: a special case tying entities, keys, and relationships together – **weak entities**.

Weak Entities and Identifying Relationships

Strong vs. Weak Entities

- **Strong entity**: has its own identifying attribute, exists independently – everything so far
- **Weak entity**: no attribute of its own is guaranteed unique – identified only by combining a **partial key** (or discriminator) with its **owner** (identifying) entity

Copy is ours: copy_number is only unique **within one book** – books 1 and 2 can both have a “copy 1.” Copy means nothing without knowing which Book it belongs to.

Why Do We Need Weak Entities?

Why not just give every `Copy` a global `copy_id`? Many real systems do, as a shortcut. But weak entities capture something true: a copy's identity is **inherently relative** to its book, like “chapter 3” only means something once you say which book. Modelling that dependency explicitly, instead of hiding it behind an arbitrary ID, is the point.

Identifying Relationships

The relationship connecting a weak entity to its owner: an **identifying relationship**.

- The weak entity has **total** participation – a Copy cannot exist without its Book
- The weak entity's identity depends on it – `copy_number` only disambiguates combined with the owning book

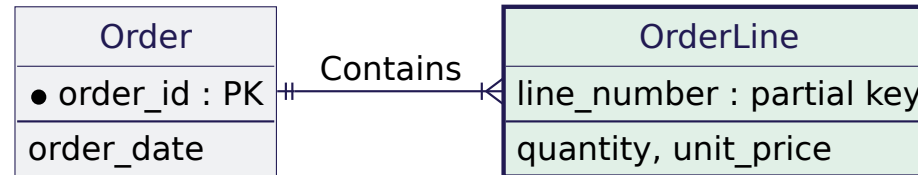
Notation: a **heavier-bordered, distinctly coloured rectangle** (a literal double border in Chen notation – seen shortly). Its partial key has no underline – alone, it does not uniquely identify anything.

Worked Example: Order and OrderLine

A second classic example: an `Order` has several `OrderLine` entries. `line_number` is only unique **within one order** – order 1's line 1 and order 2's line 1 are different rows. Exactly the same shape of problem as `Copy` and `Book`.

- `Order` (strong) – `order_id` (PK), `order_date`
- `OrderLine` (weak) – `line_number` (partial key), `quantity`, `unit_price`
- `Contains` – the identifying relationship, 1:N, and we take every order to require at least one line

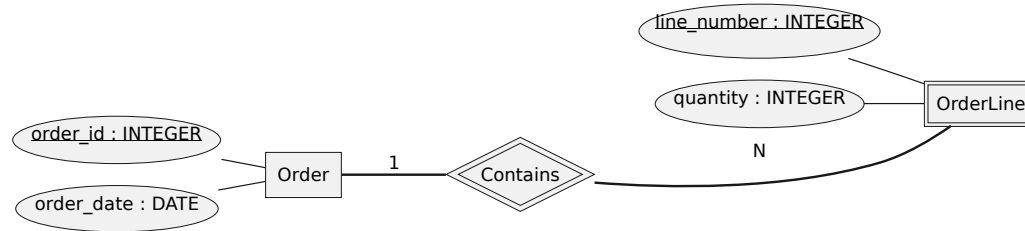
Order-OrderLine ER Diagram



The heavier border and shading mark OrderLine as weak; Contains is its identifying relationship

Both ends are mandatory: every line needs exactly one order, and every order needs at least one line (our assumption). The standard shape of an identifying relationship: strong entity on one end, weak entity on the other, both total.

Order-OrderLine: the Same Fact, in Chen Notation

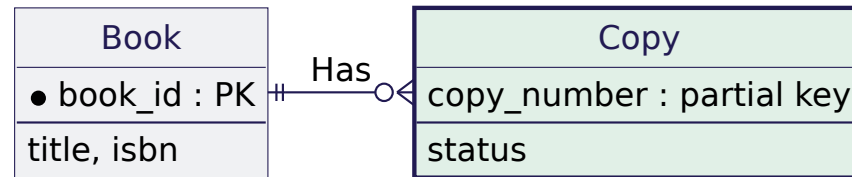


OrderLine's double-outlined box and Contains's double-outlined diamond are Chen's literal notation for a weak entity and its identifying relationship

- The double outline is not a metaphor – Chen notation literally draws it doubled, on both the weak entity and its identifying relationship
- Same fact as the crow's-foot diagram on the previous slide: both ends total, OrderLine weak

Weak Entities in Our Library: Copy

Back to the library. Copy is identified by the pair (book_id, copy_number) – book_id reaches across the identifying relationship from Book; copy_number alone only disambiguates copies of the **same** book.



Has is Copy's identifying relationship – notice Book stays partial, unlike the Order/OrderLine example

Unlike Order/OrderLine: here Book's participation is **partial** – a book can sit with zero copies in stock. Copy is still weak either way – what matters is **Copy's** total participation and partial key, not the owner's side.

Weak Entity Notation: Recap

- **Weak entity** → heavier-bordered rectangle (double border in Chen notation); no attribute of its own is globally unique
- **Partial key** → unique only within one owner instance (`copy_number` within one Book)
- **Identifying relationship** → connects a weak entity to its owner; the weak entity's participation is always total
- A weak entity's real key, once we reach the relational model later in the course, becomes a **composite** key: owner's key plus partial key together

ER vs. UML and the Live Demo

ER Diagrams vs. UML Class Diagrams

You may know **UML class diagrams** from an OO or software-engineering course. They look strikingly similar to crow's-foot ER diagrams – both describe “things,” their properties, and their relations. The difference: ER models **persistent data**; UML models **software structure**, including behaviour.

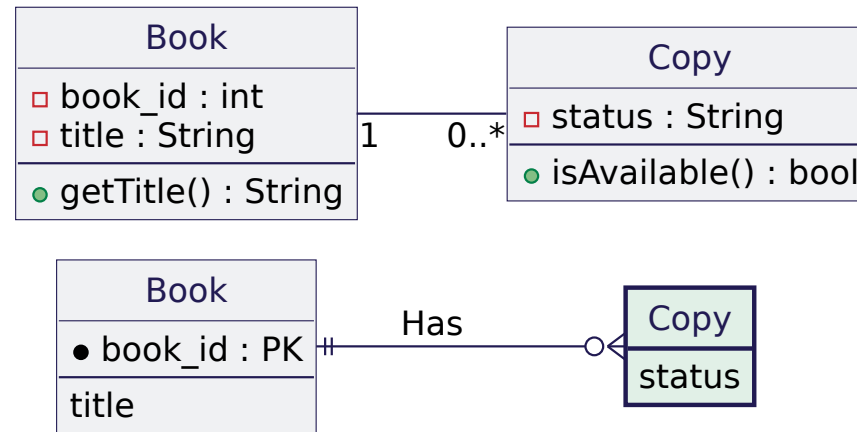
Similarities

- Both draw a “thing” as a rectangle – **entity** in ER, **class** in UML
- Both list properties inside the box – **attributes** in ER, **attributes/fields** in UML
- Both annotate association lines with multiplicity – crow’s-foot symbols correspond directly to UML labels like 1, 0..*
- Both can mark one association end optional and the other mandatory

Differences

- UML class boxes add a **third compartment** for **operations** – ER entities hold only data, no behaviour
- UML multiplicities are **text** (1, 0..1, 0..*, 1..*), not crow's-foot symbols
- ER's **weak entity** has no exact UML equivalent – closest is **composition** (filled-diamond association)
- UML supports **inheritance** between classes; classical ER has no standard notation for it (some extensions add it, outside this course's scope)

Side-by-Side: Book and Copy in Both Notations



Top: UML class diagram, with an operations compartment and text multiplicities. Bottom: the same pair in ER / crow's foot

ER vs. UML: Comparison Table

ER term	UML term
Entity / entity type	Object / class
Attribute	Attribute (field)
Relationship	Association
Cardinality (crow's foot)	Multiplicity (text, e.g. 0..*)
Weak entity + identifying relationship	Composition (closest analogue, not identical)
- (no equivalent)	Operations / methods, inheritance

Tools for Drawing ER Diagrams

Two tools are enough for your exercise and assignment:

- **draw.io** (diagrams.net) – free, browser or desktop, dedicated ER shapes with crow's-foot connectors
- **PlantUML** – text-based; you write a short script and render it, either locally (- Playout=smetana, no Graphviz needed) or with the online server – exactly what produced today's own diagrams
- PlantUML's entity/crow's-foot syntax matches today's diagrams – a good pick for the live demo, and it is what the exercise sheet's extras ship as, already rendered for you

PlantUML Crow's-Foot Syntax Preview

Just a taste – exercise materials cover this in full. PlantUML spells out the same crow's-foot symbols as characters instead of drawing them:

```
entity Book {
    * book_id : int <<PK>>
    --
    title : string
    isbn : string
}
entity Copy {
    * copy_number : int <<partial key>>
    --
    status : string
}
Book ||--o{ Copy : Has
```

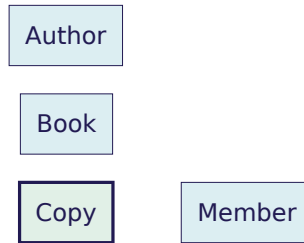
|| = “exactly one,” o{ = “zero or many” – the same bar/circle/fork vocabulary as the crow's-foot legend, just written left-to-right.

Live Demo: Modelling the Library System

We build the library ER diagram together, live, in draw.io or PlantUML. One piece at a time: entities, then attributes, then each relationship, checking cardinality and participation as we go. The next four slides show where we're headed.

Building the Diagram, Step 1: Entities

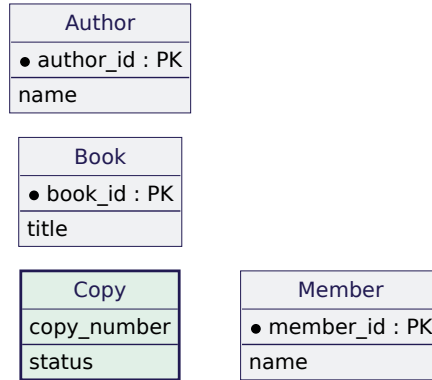
Just the boxes first – no attributes, no relationships. Four entity types, one already known to be weak.



Four entity types: Author, Book, Copy (weak), Member – no attributes or relationships yet

Step 2: Add Attributes

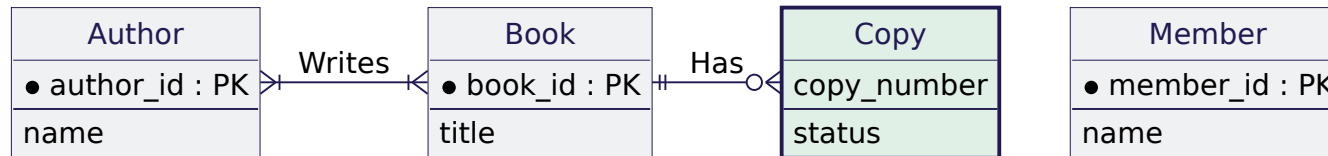
Next, fill in each entity's key and a couple of its main attributes.



Attributes added – still no relationships drawn between the boxes

Step 3: Add Writes and Has

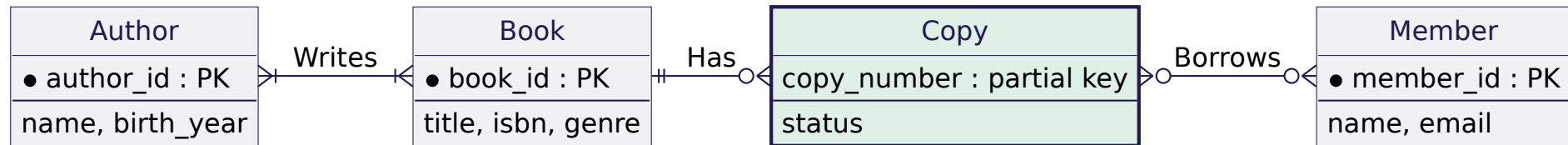
Now connect Book to Author (M:N, total both sides) and Book to Copy (its identifying relationship).



Writes (M:N, total both sides) and Has (1:N, Copy's identifying relationship) added

Step 4: Add Borrows – the Complete Diagram

Finally, connect Member to Copy. The result: our full library ER diagram, everything today built toward.



The complete library ER diagram: 4 entities, 3 relationships, all cardinalities and participation marked

Borrows also carries `loan_date`, `due_date`, `return_date`, and derived `is_overdue` – omitted here to keep the diagram readable, as discussed earlier.

Wrap-Up

Java Connection: From ER to Classes

Language-agnostic – Java, Python, C# all work the same way – but since Java is this course's concurrent track, here's the correspondence:

- **Entity type** → **class**: Book, Copy, Author, Member each become a class, attributes become fields
- **Relationship** → **reference or collection**: Book's Has toward Copy becomes a List<Copy> field; an M:N relationship like Writes typically becomes a collection on **both** sides
- **Weak entity's dependency on its owner** → a constructor or field requirement – no Copy object without a Book reference

We revisit this concretely with the **Repository** and **DAO** patterns later in the course.

Today: entities and entity types, the four kinds of attributes, relationships (cardinality, participation), weak entities and identifying relationships, and the UML mapping.